
EMPLOYMENT MANAGEMENT SYSTEM

Attendance

Check-in, check-out, search, and the self-service endpoint

ATTENDANCE FLOWS AND STATUS RULES • 2 OCTOBER 2026

Contents

01	1. The domain in one picture	3
02	2. The status enum	3
03	3. DTOs are Java <code>records</code>	4
04	4. Flow A - Check in	4
05	5. Flow B - Check out	7
06	6. Flow C - Manager search	8
07	7. <code>AttendanceSpecification</code> - the Criteria API	10
08	8. Flow D - An employee's own records (<code>/me</code>)	10
09	9. The mapper	11
10	10. <code>AttendancePage</code> - the whole page in one place	12
11	11. How attendance history gets created without anyone clocking in	13
12	12. Attendance endpoint summary	13

The only feature employees can use. This is where the whole role model pays off, and it contains the most interesting business logic in the app.

1. The domain in one picture

```

POST /api/attendance/check-in?employeeId=5
|
v
is there already a row (employee 5, today)?
no  → INSERT with checkIn = now, status = INCOMPLETE
yes → if checkIn ≠ null → 400 "Employee already checked in today"
     else                → set checkIn = now, status = INCOMPLETE

POST /api/attendance/check-out?employeeId=5
|
v
row for (5, today) exists and checkIn ≠ null? else 400
|
v
checkOut    = now
totalHours  = round2( minutes(checkIn → checkOut) / 60 )
status      = totalHours < 4      → HALF_DAY
              checkIn > 09:00    → LATE
              otherwise          → PRESENT
  
```

`LATE_CUTOFF = LocalTime.of(9, 0)` and `HALF_DAY_MIN_HOURS = 4.0` are `static final` constants at the top of `AttendanceServiceImp` (lines 28-29). Note `checkIn.isAfter(LATE_CUTOFF)` - exactly 09:00:00 is **not** late; anything after it is.

The order matters: a short day that also started late is classified `HALF_DAY`, not `LATE`, because the hours check returns first.

2. The status enum

```

public enum AttendanceStatus { PRESENT, INCOMPLETE, LATE, HALF_DAY, ABSENT }
  
```

Status	Meaning	How it is set
INCOMPLETE	checked in, not yet checked out	checkIn
PRESENT	full day, on time	checkOut
LATE	full day, after 09:00	checkOut

Status	Meaning	How it is set
HALF_DAY	fewer than 4 hours, regardless of start time	checkOut
ABSENT	no check-in at all	only by DataSeeder.backfillAbsent

`@Enumerated(EnumType.STRING)` on the entity means the database stores the text `"PRESENT"`, not a number. That makes the SQL readable and means reordering the enum constants would not corrupt data.

Important: `ABSENT` can never be produced by the API. There is no "mark absent" endpoint. Absent rows are created at startup for past weekdays.

3. DTOs are Java records

```

public record AttendanceSearchReq(Long employeeId, Long departmentId,
                                  LocalDate startDate, LocalDate endDate,
                                  AttendanceStatus status) { }

public record AttendanceResponse(Long id, Long employeeId, String employeeName,
                                  LocalDate date, LocalDateTime checkInTime,
                                  LocalDateTime checkOutTime, AttendanceStatus status) { }

```

JAVA

Records are immutable, have no setters, and generate accessors as `employeeId()` rather than `getEmployeeId()`. Jackson deserialises records by constructor, so the JSON field order does not matter. Note that the service reads `criteria.employeeId()`, `criteria.status()` - no `get` prefix.

4. Flow A - Check in

Endpoint: `POST /api/attendance/check-in?employeeId=5` -> `200`

Note there is **no request body**. `http.js:100`:

```

export async function checkIn(employeeId) {
  return api(`/api/attendance/check-in?employeeId=${employeeId}`, { method: "POST" });
}

```

JS

Because `options.body` is absent, `api()` does not set a `Content-Type` header - which is why the call works. (A `curl -X POST` that forces `application/x-www-form-urlencoded` will fail, because Spring rejects the content type and no handler exists for `HttpMediaTypeNotSupportedException`, so it becomes a generic `500`.)

Authorization - the cleverest annotation in the codebase

```
@PreAuthorize("hasAnyRole('ADMIN','MANAGER') or " +
    "hasRole('EMPLOYEE') and " +
    "authentication.principal.employee ≠ null and " +
    "#employeeId == authentication.principal.employee.id")
```

This is Spring Expression Language (SpEL) evaluated against the method arguments and the authentication. Reading it in order:

Fragment	Meaning
<code>hasAnyRole('ADMIN','MANAGER')</code>	managers may clock anyone - short-circuit <code>true</code>
<code>or hasRole('EMPLOYEE')</code>	otherwise the caller must be an employee
<code>and authentication.principal.employee ≠ null</code>	and must actually have a linked employee record (a manager-only account has none)
<code>and #employeeId == authentication.principal.employee.id</code>	and must be asking for themselves

`#employeeId` is the **method parameter name** - SpEL exposes `@RequestParam` parameters as `#name`. `authentication.principal` is the `User` entity the JWT filter put in the context (because `User` implements `UserDetails`). So the comparison reads directly off the entity: does the requested id equal *my* linked employee id?

This means an employee cannot clock a colleague even if they hand-edit the URL - the check compares against the token's account, not against anything the client controls beyond the target id, which must match their own.

OperatorPrecedence in SpEL: `and` binds tighter than `or`, so the expression is `(ADMIN or MANAGER) or (EMPLOYEE and employee ≠ null and #employeeId == own id)`. That is the intended grouping.

The service logic

```
Employee employee = employeeRepository.findById(employeeId)
    .orElseThrow(() → new InvalidInputException("Employee not found"));
```

A missing employee is a **400** (`InvalidInputException`), not a 404 - defensible but inconsistent with `ResourceNotFoundException` elsewhere.

```
LocalDate today = LocalDate.now();
Attendance attendance = attendanceRepository.findByIdAndDate(employeeId, today)
    .orElseGet(() → {
        Attendance newAttendance = new Attendance();
        newAttendance.setEmployee(employee);
```

```
newAttendance.setDate(today);
return newAttendance;
});
```

`orElseGet` (not `orElse`) because the supplier builds a new object and should only run when nothing was found. Note this means check-in **creates the row** if the seeder has not already written today's `ABSENT` placeholder... and if it *has* written one, the existing row is reused and overwritten - which is exactly how a seeded `ABSENT` row for today becomes an `INCOMPLETE` one after check-in.

```
if (attendance.getCheckIn() != null) {
    throw new InvalidInputException("Employee already checked in today");
}
```

JAVA

The double-check-in guard. **400**.

```
attendance.setCheckIn(LocalTime.now());
attendance.setCheckOut(null);
attendance.setTotalHours(null);
attendance.setStatus(AttendanceStatus.INCOMPLETE);
return attendanceMapper.toDtoResponse(attendanceRepository.save(attendance));
```

JAVA

- `checkIn` is a `LocalTime` (column `TIME`), not a timestamp - which is why the mapper later combines it with `date` to make a full `LocalDateTime`.
- `setCheckOut(null)`, `setTotalHours(null)` and `INCOMPLETE` reset any previous state, so re-running the seeder's flow for today cannot leave stale data.
- `save` issues the `INSERT` / `UPDATE`, and the saved entity is mapped and returned.

The frontend button logic

`AttendancePage.jsx:180-205` decides which of three UI states to render from **one extra query**, not from the main table:

```
const { data: todayData } = useQuery({
  queryKey: ["attendance-today", canManage ? "search" : "me", effectiveClockEmployeeId],
  queryFn: () => canManage
    ? searchAttendance({ employeeId: effectiveClockEmployeeId, startDate: today, endDate: today }, 0, 1)
    : fetchMyAttendance(0, 1, today, today),
  enabled: canManage ? !!effectiveClockEmployeeId : true,
});
const todayRow = todayData?.content?.[0];
```

JSX

```
{todayRow?.checkInTime && !todayRow.checkOutTime ? /* Checked in at HH:MM + Check Out button */
  : todayRow?.checkInTime && todayRow.checkOutTime ? /* "Done: present" */
  : /* Check In button */}
```

JSX

`size=1` and `today` as both `startDate` and `endDate` narrow it to at most one row. Note the two branches **fetch from different endpoints** depending on role - that is the point of the whole feature: an employee never touches `/search`.

`today` is computed as `new Date().toISOString().slice(0, 10)` - UTC, not local. For timezones far from UTC this can be the wrong day; a `LocalDate.now()` on the server may then disagree with the frontend's "today". A real bug in non-UTC timezones.

5. Flow B - Check out

Endpoint: `POST /api/attendance/check-out?employeeId=5`, same `@PreAuthorize`.

```
if (!employeeRepository.existsById(employeeId)) {
    throw new InvalidInputException("Employee not found");
}
```

JAVA

`existsById` rather than `findById` - the employee object is not needed, only existence. (`checkIn` needs the object because it has to assign the relationship.)

```
Attendance attendance = attendanceRepository.findByIdAndDate(employeeId, today)
    .orElseThrow(() -> new InvalidInputException("Employee has not checked in today"));

if (attendance.getCheckIn() == null) {
    throw new InvalidInputException("Employee has not checked in today");
}
```

JAVA

Two identical guards for two different situations: no row for today at all, or a row that exists but has no check-in (a seeded `ABSENT` row). Both **400**. There is no separate "already checked out" error - checking out twice simply overwrites `checkOut` and recomputes.

```
LocalTime checkOut = LocalTime.now();
attendance.setCheckOut(checkOut);
double totalHours = Math.round(
    Duration.between(attendance.getCheckIn(), checkOut).toMinutes() / 60.0 * 100.0) / 100.0;
attendance.setTotalHours(totalHours);
attendance.setStatus(computeStatus(attendance.getCheckIn(), checkOut, totalHours));
```

JAVA

Reading the arithmetic:

1. `Duration.between(checkIn, checkOut)` -> a duration in minutes.
2. `/ 60.0` -> hours. The `.0` forces floating-point division, so `toMinutes()` (a `Long`) is not truncated to an integer number of hours.
3. `* 100.0` then `Math.round(...)` then `/ 100.0` - the standard round-to-two-decimals trick, so 8h29m becomes `8.48`, not `8.483333333333333`.

4. `computeStatus( ... )` as in section 1.

`totalHours` is written to the database but **is not in** `AttendanceResponse` - the frontend never receives it. It is effectively write-only (only the seeder's 8.5 and this calculation ever populate it, and nothing reads it back).

computeStatus (line 126)

```
private AttendanceStatus computeStatus(LocalTime checkIn, LocalTime checkOut, double totalHours) {
    if (totalHours < HALF_DAY_MIN_HOURS) return AttendanceStatus.HALF_DAY;
    if (checkIn.isAfter(LATE_CUTOFF)) return AttendanceStatus.LATE;
    return AttendanceStatus.PRESENT;
}
```

- `checkOut` is a parameter but is **never used** - the decision is made from `totalHours`, which already encodes the span. Dead parameter, kept for clarity.
- `INCOMPLETE` is never returned here; it belongs to check-in.
- Edge case: a check-in at 23:59 and a check-out at 00:05 next day gives a **negative** duration, so `totalHours < 4` is true and the day is `HALF_DAY`. Overnight shifts are not handled.

6. Flow C - Manager search

Endpoint: `POST /api/attendance/search?page=0&size=10`, body is `AttendanceSearchReq` (or absent).

```
@PreAuthorize("hasAnyRole('ADMIN','MANAGER')")
@PostMapping("/search")
public ResponseEntity<Page<AttendanceResponse>> searchAttendance(
    @RequestBody(required = false) AttendanceSearchReq req, Pageable pageable){
    return ResponseEntity.ok(attendanceService.searchAttendance(req, pageable));
}
```

- `@RequestBody(required = false)` - a missing body is legal, not a `400`.
- `POST` (not `GET`) because the filter set is a structured object, and because `{"departmentId": ... }` reads better than five query params.
- No `@Valid`, and the record has no constraints - so bad values (an unknown status string) surface as Jackson errors -> `400 Malformed request body`.

The service, and why it looks the way it does

```
// The controller accepts a missing body (required = false), so normalise
// null to an empty filter set - otherwise req.employeeId() below NPEs.
AttendanceSearchReq criteria = (req == null)
```

```
? new AttendanceSearchReq(null, null, null, null, null)
: req;
```

This is a real bug fix. Sending `POST /api/attendance/search` with the JSON body `null` (or with no body at all) used to reach `req.employeeId()` on a `null` reference and produce a `500`. The normalisation makes "no filters" mean "all rows". **Verified: `null` now returns `200` with 45 rows.**

```

List<Specification<Attendance>> filters = new ArrayList<>();
if (criteria.employeeId() != null) filters.add( ... hasEmployeeId( ... ));
if (criteria.departmentId() != null) filters.add( ... hasDepartmentId( ... ));
if (criteria.status() != null) filters.add( ... hasStatus( ... ));
if (criteria.startDate() != null && criteria.endDate() != null)
    filters.add( ... dateBetween( ... ));

Specification<Attendance> spec = Specification.allOf(filters);
return attendanceRepository.findAll(spec, pageable).map(attendanceMapper::toDtoResponse);

```

Four things to be able to explain:

1. `if (x != null)` around every filter - a null filter means "do not filter", not "match null".
2. The date filter needs BOTH bounds. Sending only `startDate` filters nothing at all. That is a silent-ignore trap: a user picking just a "From" date gets unfiltered results with no warning.
3. `Specification.allOf(filters)` instead of `Specification.where(null)`. The comment says it plainly: `Specification.where` is forbidden in Spring Data JPA 4 and throws "Specification must not be null". Building a list and calling `allOf` handles the empty-list case correctly (an empty `allOf` matches everything).
4. `.map(attendanceMapper::toDtoResponse)` converts each element of the `Page`.

@Transactional(readOnly = true) here is load-bearing

```

// NOTE: readOnly tx keeps the Hibernate session open so the LAZY
// Attendance.employee proxy can be loaded by the mapper below.
// Without it the session closes after findAll() and the mapper
// throws LazyInitializationException -> 500 "unexpected error".

```

`AttendanceResponse.employeeName` comes from `attendance.getEmployee().getFullName()`, and `Attendance.employee` is `FetchType.LAZY`. Reading it outside a transaction throws `LazyInitializationException`, which `GlobalExceptionHandler` catches as the generic `Exception` -> `500`. The `readOnly` transaction is therefore not an optimisation, it is required for correctness.

7. AttendanceSpecification - the Criteria API

```
public static Specification<Attendance> hasEmployeeId(Long employeeId){
    return (root, query, cb) -> cb.equal(root.get("employee").get("id"), employeeId);
}
```

JAVA

A `Specification` is a lambda over three Criteria-API objects:

- `root` - the `Attendance` "root" (from alias `a`),
- `query` - the query being built (used for subqueries; unused here),
- `cb` - the criteria builder, which creates the predicates.

`root.get("employee").get("id")` **navigates the association**, so the generated SQL contains an implicit `join`:

```
select ... from attendances a
join employees e on a.employee_id = e.id
where a.employee_id = ?
```

SQL

`hasDepartmentId` navigates two levels: `root.get("employee").get("department").get("id")`, producing a second join through `departments`. This is exactly why Specifications are used here instead of a hand-written `@Query`: the joins follow the object graph without any SQL to maintain.

`dateBetween` uses `cb.between(root.get("date"), start, end)` -> `date between ? and ?`, and it is **inclusive** on both ends.

8. Flow D - An employee's own records (/me)

Endpoint: `GET /api/attendance/me?page=0&size=10&startDate=&endDate=`

```
@PreAuthorize("hasRole('EMPLOYEE')")
@GetMapping("/me")
public ResponseEntity<Page<AttendanceResponse>> myAttendance(
    @AuthenticationPrincipal User user,
    @RequestParam(required = false) @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate
    startDate,
    @RequestParam(required = false) @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate
    endDate,
    Pageable pageable){

    AttendanceSearchReq req = new AttendanceSearchReq(
        user.getEmployee().getId(), null, startDate, endDate, null);
    return ResponseEntity.ok(attendanceService.searchAttendance(req, pageable));
}
```

JAVA

Key points:

- `hasRole('EMPLOYEE')` **only**. A manager calling `/me` gets `403`. The endpoint is defined for employees; managers use `/search`. (This is intentional and asymmetric - see the note at the end of this document.)
- `@AuthenticationPrincipal User user` is Spring's shorthand for `authentication.getPrincipal()`, type-cast to `User`. Because `User implements UserDetails`, the principal really is the entity.
- **The employee id is taken from the token, never from the request.** There is no `employeeId` parameter at all. That is the guarantee the comment describes: an employee cannot widen the scope by tampering with the request, because there is nothing to tamper with.
- The optional dates get `@DateTimeFormat(iso = ISO.DATE)` so `2024-01-15` converts to a `LocalDate`. Without it, Spring would try a full date-time.
- The controller **builds the same** `AttendanceSearchReq` as `/search` and calls the same service method. No duplicated filtering logic.
- `user.getEmployee().getId()` would NPE if the account had no employee record. The `@PreAuthorize` on `/check-in / check-out` guards against that case with `authentication.principal.employee != null`, but `/me` **has no such guard** - an `ROLE_EMPLOYEE` account with `employee_id = NULL` would get a `500` here. The seeder never creates such an account, and registration always links one, so it is unreachable in practice.

9. The mapper

```

@Mapping(target = "employeeId", source = "employee.id")
@Mapping(target = "employeeName", source = "employee.fullName")
@Mapping(target = "checkInTime", expression = "java(attendance.getCheckIn() != null ?
java.time.LocalDateTime.of(attendance.getDate(), attendance.getCheckIn()) : null)")
@Mapping(target = "checkOutTime", expression = "java(attendance.getCheckOut() != null ?
java.time.LocalDateTime.of(attendance.getDate(), attendance.getCheckOut()) : null)")
AttendanceResponse toDtoResponse(Attendance attendance);

```

- `source = "employee.fullName"` uses the **computed getter** `Employee.getFullName()` which returns `"Jane Smith"` - so the response is flat, no nested object.
- The two `expression` mappings solve the `LocalTime` problem. The database stores a *date* column and two *time* columns; the API returns full timestamps, so the mapper stitches them: `LocalDateTime.of(date, checkIn)`.
- The null checks matter: an `ABSENT` row has both times `null`, and `LocalDateTime.of(date, null)` would throw.
- `totalHours` is absent from `AttendanceResponse`, so MapStruct never touches it
 - which is why it can safely be a non-loaded column.

10. AttendancePage - the whole page in one place

Derived state, no effect

```
const effectiveClockEmployeeId = canManage
  ? clockEmployeeId
  : String(user?.employeeId ?? "");
```

JSX

A manager picks from the dropdown (`clockEmployeeId`, real state); an employee's id comes from the `/me` response (`user.employeeId`). It is a **plain derived value computed during render**, not stored state - so there is no `useEffect` to keep the two in sync, and no chance of them disagreeing. An earlier version synchronised `clockEmployeeId` with an effect and could briefly hold the wrong id.

`String( ... )` and `?? ""` guard against `user` being null or `employeeId` missing.

Filters vs applied filters

```
const [filters, setFilters] = useState(emptyFilters); // the inputs
const [applied, setApplied] = useState(emptyApplied); // what was submitted
```

JSX

`handleChange` updates `filters` (using `e.target.name`); `handleApply` (form submit) converts `""` to `null` and copies into `applied`, resetting `page` to 0. Because `applied` is part of the query key, **nothing is requested until Apply is pressed**. `handleReset` clears both.

The whole filter `<form>` is inside `{canManage && ( ... )}` (line 209) - employees see no filters at all, because there is nothing they could change.

One query, two endpoints

```
queryKey: ["attendance", canManage ? "search" : "me", applied, page],
queryFn: () => canManage ? searchAttendance(applied, page) : fetchMyAttendance(page),
placeholderData: (prev) => prev,
```

JSX

- The role is part of the query key, so a role change can never show cached rows from the other scope.
- `placeholderData: (prev) => prev` keeps the previous page visible while the new one loads instead of flashing "Loading...". `isApplying = isFetching && !isLoading` distinguishes "first load" from "refetch", so the table shows a loading label only on the first load.
- Employee column and the "Clock In/Out" heading both switch on `canManage`.

Mutation feedback

```
onSuccess: (data) => {
  pushToast(`Checked in at ${formatTime(data.checkInTime)}`);
  invalidateAttendance();
}
```

JSX

`formatTime(value)` { `return value ? value.slice(11, 16) : "-"` } slices characters 11-16 out of an ISO string like `"2024-05-01T09:12:33"` -> `"09:12"`. It is a string slice, not date formatting, so it assumes ISO-8601 without a timezone suffix.

`invalidateAttendance()` invalidates three keys: `["attendance"]`, the main table, `["attendance-today"]`, the button state, and `["dashboard-attendance"]`, the dashboard counts. All three because one clock action changes three views.

11. How attendance history gets created without anyone clocking in

`DataSeeder.backfillAbsent` (explained in `01-foundations.md` section 8) runs on every startup and inserts `ABSENT` rows for the previous 14 weekdays where no row exists, guarded by `attendanceRepository.existsByEmployeeIdAndDate(employee.getId(), date)`.

So the demo database fills up automatically over time. Two consequences worth knowing:

- **Check-in reuses a seeded `ABSENT` row for today.** `checkIn` looks the row up by `(employee, today)` and finds the backfilled `ABSENT` one, then overwrites its fields. That is why `checkIn` sets `checkOut = null` and `totalHours = null` explicitly.
- **The seeder never writes today's row**, because it only walks `day = 1..14` backwards. Today's row is created only by an actual check-in.

12. Attendance endpoint summary

Method	Path	Access	Params / Body	Success
POST	<code>/api/attendance/search</code>	ADMIN, MANAGER	body <code>AttendanceSearchReq</code> (optional), <code>page</code> , <code>size</code>	200 <code>Page<AttendanceResponse></code>
GET	<code>/api/attendance/me</code>	EMPLOYEE	<code>page</code> , <code>size</code> , <code>startDate?</code> , <code>endDate?</code>	200 <code>Page<AttendanceResponse></code>

Method	Path	Access	Params / Body	Success
POST	<code>/api/attendance/check-in</code>	managers any; employees only themselves	<code>employeeId</code>	200 AttendanceResponse
POST	<code>/api/attendance/check-out</code>	managers any; employees only themselves	<code>employeeId</code>	200 AttendanceResponse

Error cases:

Situation	Status	Message
employee calls <code>/search</code>	403	Forbidden: insufficient permissions
manager calls <code>/me</code>	403	Forbidden: insufficient permissions
employee clocks someone else	403	Forbidden: insufficient permissions
unknown employee id	400	Employee not found
check in twice	400	Employee already checked in today
check out with no check-in	400	Employee has not checked in today
malformed JSON body	400	Malformed request body

Known asymmetry: an `EMPLOYEE` cannot read their own records through `/search`, and a `MANAGER` cannot read their own through `/me`. That was a deliberate tightening so each role has exactly one read path - but it means a manager with a linked employee record has no "my attendance" view at all. If you ever want one, the fix is `hasAnyRole('EMPLOYEE', 'ADMIN', 'MANAGER')` on `/me`; the employee id already comes from the token, so no new leak is created.